

Lab 2 Interview Guide

Lab 2 — Interview preparation guide

Purpose: explain what each question asked, what the files in this folder implement, and why the implementation works.

Prepared from your local assignment, Python files, notebook, notes and saved results. First-person answers below are rehearsal scripts describing those files; they do not assume a particular development history. No Lab 2 submission report PDF was found, so the guide cannot confirm the exact contents of the report you uploaded.

1. Your opening explanation

“The lab covered three approaches to sequential decision-making. First, I used value iteration to plan with a known model of the environment. Then I implemented tabular Q-learning, which learns action values from experience, and used it in Gridworld, Crawler and Pacman. Finally, for Question 9, I chose policy search and implemented REINFORCE to learn a softmax policy for CartPole directly.”

Part	Your implementation	What it learns or computes
Q1	reinforcement/valueIterationAgents.py	State values and a greedy policy using the known MDP
Q2–Q3	reinforcement/analysis.py	Parameter choices producing particular policies
Q4–Q5	reinforcement/qlearningAgents.py	A table of state-action values; epsilon-greedy action selection
Q6	analysis.question6()	Bridge exploration analysis: NOT POSSIBLE
Q7	Existing PacmanQAgent using your Q-learning methods	A policy for complete Pacman board states
Q8	Optional; methods still contain placeholders	Approximate Q-learning was not implemented
Q9(b)	826685_875455_policy_search.ipynb	A parameterized CartPole policy trained with REINFORCE

The notebook has its own “Question 1” and “Question 2”. Both belong to the assignment’s **Question 9(b)**. You selected policy search, not the Gaussian-process alternative.

2. Concepts you should be able to explain

An MDP describes states, legal actions, transition probabilities, rewards and a discount factor. A policy specifies how to choose an action in each state.

Symbol or parameter	Meaning	Where it appears in your work
$V(s)$	Expected discounted return from a state under a specified policy, or its optimal value	Value iteration stores estimates in <code>self.values</code>
$Q(s, a)$	Expected discounted return when taking action a in state s , then following a policy	Q-learning stores estimates in <code>self.Qvalues</code>
γ	Discount: how much future rewards count	<code>self.discount</code> ; notebook experiment uses <code>1.0</code>
α	Learning rate: size of a learning update	Q-learning's <code>self.alpha</code> ; optimizer learning rate in <code>CartPole</code>
ϵ	Probability of choosing an exploratory random action	<code>getAction</code> in Q-learning
Noise	Probability of unintended movement after selecting an action	Gridworld transition model
Living reward	Reward for leaving an ordinary non-exit grid cell	Q3 parameter tuning

Do not confuse epsilon and noise. Epsilon changes which action the agent chooses. Noise changes what the environment does after that choice. A greedy agent can still move unpredictably in a noisy environment.

In this Gridworld, a numbered reward square is **pre-terminal**: the agent enters it, then takes the special `exit` action to receive its exit reward and reach the true terminal state. The true terminal state has no actions and value zero.

3. Question 1 — Value iteration

What was required

Implement a planner for a known MDP, using **batch value iteration**, and provide functions for action values and the greedy policy. This question was intended to be coded during the lab with the instructor.

The Bellman optimality update is:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_k(s')].$$

It says: evaluate each possible action by averaging over its possible outcomes, then keep the best action value.

How your code implements it

1. The constructor stores the MDP, discount and iteration count. `util.Counter()` initializes all state values to zero by default.
2. `runValueIteration()` creates a fresh `new_values` table for each sweep.
3. For each state, `getAction(s)` finds the action with the highest one-step lookahead value.
4. `getQValue(s, a_star)` computes that action's value and stores it in the new table.
5. Only after the whole sweep does `self.values = new_values` replace the old table.

That last step makes the update **batch/synchronous**: every calculation in a sweep uses the previous sweep's values. Updating `self.values` inside the state loop would use a mixture of old and new values and make finite-iteration results depend on state order.

`computeQValueFromValues(state, action)` implements:

$$Q_{k+1}(s, a) = \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_k(s')].$$

It loops through `getTransitionStatesAndProbs`, multiplies each reward-plus-future-value term by its probability, and adds the results. For a terminal state it returns zero.

`computeActionFromValues(state)` implements:

$$\pi_{k+1}(s) = \operatorname{argmax}_a Q_{k+1}(s, a).$$

Your code checks all available actions and keeps the first maximum because it replaces the current choice only on a strict `>` comparison. Terminal states return `None`. There is no exploratory action selection in this planner.

Small numerical example: if one action has outcomes with probabilities 0.8 and 0.2, immediate rewards 0 and -10, successor values 5 and 0, and discount 0.9, its value is $0.8(0 + 0.9 \times 5) + 0.2(-10 + 0) = 1.6$.

What to say aloud

"I decomposed the Bellman update into an action-value calculation and an argmax over actions. I used a separate table for the next iteration, so all states use the same previous value function. This is planning: it uses transition probabilities and rewards from the MDP instead of learning them through interaction."

Possible follow-ups:

- **Why do displayed Q-values look one step further ahead?** After storing V_k , computing a Q-value adds an immediate reward and then uses V_k , giving a $k + 1$ -reward lookahead.
- **Does it stop at convergence?** Your implementation runs the specified number of sweeps; it does not test a convergence tolerance.
- **Any limitation?** The policy function assumes a nonterminal state has at least one action. It handles the provided terminal states, but a more general implementation would also explicitly check for an empty action list.

4. Question 2 — Crossing the bridge

What was required

Change **only one** of discount or noise so that value iteration chooses to cross `BridgeGrid` toward the larger reward. Defaults were discount 0.9 and noise 0.2.

Your answer and explanation

```
answerDiscount = 0.9
answerNoise = 0.01
```

You kept the discount unchanged and reduced noise. BridgeGrid has a nearby +1 exit, a distant +10 exit, and -100 chasms beside the bridge. Lower noise reduces the probability of unintended movement into those chasms, making the high-reward route attractive in expected discounted return.

“I changed only the transition noise, from 0.2 to 0.01. At the same discount, the distant reward becomes worthwhile because crossing is much more reliable. The policy attempts the bridge; this does not mean every execution succeeds.”

This is a change to the **environment model**, not an exploration setting or a learning-rate adjustment.

5. Question 3 — Producing five different policies

What was required

Choose (discount, noise, living reward) for each requested behavior on DiscountGrid. The policy is planned with the specified noise, but the required behavior is judged along its **noiseless path**.

Part	Requested behavior	Your exact tuple	Main explanation
3(a)	Close +1 exit, near cliff	(0.2, 0.001, -0.2)	Strong discounting favors the close reward; tiny noise and a step penalty favor the short route
3(b)	Close +1 exit, safe route	(0.2, 0.1, 0.1)	Strong discounting favors the close reward; more noise makes cliff-adjacent moves costly; positive living reward supports the detour
3(c)	Distant +10 exit, near cliff	(0.95, 0.001, -0.2)	Future reward retains its value; tiny noise and a step penalty favor a direct route
3(d)	Distant +10 exit, safe route	(0.95, 0.1, 0.2)	High discount preserves the distant payoff; noise discourages the cliff; living reward supports the longer route
3(e)	Avoid exits and cliff	(0.01, 0.2, -0.2)	An extremely short planning horizon and the noisy model yield a cycle on the checked noiseless path

These effects interact. A positive living reward does not always make the agent exit, and a high discount does not by itself guarantee selection of the distant exit. The resulting Bellman values determine the final policy.

The important subtlety in 3(e)

Your living reward is **negative**. Therefore, do not say “I rewarded the agent for staying alive.” That would contradict your code.

With these exact settings, 100 value-iteration sweeps give:

```
(0, 1): choose north -> (0, 2)
(0, 2): choose south -> (0, 1)
```

On a noiseless execution this repeats without reaching an exit. The discount of 0.01 almost eliminates the influence of distant rewards, while the modeled movement risk affects nearby action comparisons. At (0, 2), the competing action values are extremely close: around -0.202020202. This is an unusual, numerically delicate solution, rather than a broadly robust way to encourage indefinite survival.

The current Q3 tests pass all five choices. That confirms the specified path behavior; it does **not** establish that the agent survives forever when actual transition noise is present.

“For the first four cases I used discount to influence the preferred exit and noise plus living reward to influence the route. My fifth case is less intuitive: a tiny discount leads to a two-state cycle under noiseless execution, even with a negative living reward. It passes the requested path test, but I would not claim guaranteed survival under actual noise.”

6. Question 4 — Tabular Q-learning

What was required

Learn action values from individual experiences (state, action, nextState, reward), without needing the transition probabilities used by value iteration.

Your representation and methods

`self.Qvalues = util.Counter()` stores a value for each (state, action) pair. Unseen pairs return zero.

Method	What your implementation does
<code>getQValue(s, a)</code>	Reads <code>self.Qvalues[(s, a)]</code>
<code>computeValueFromQValues(s)</code>	Returns the largest Q-value among all legal actions , or zero if none exist
<code>computeActionFromQValues(s)</code>	Collects every maximizing action and chooses randomly among them; returns <code>None</code> if there are no actions
<code>update(s, a, s_next, reward)</code>	Applies one temporal-difference update to the experienced state-action pair

Your update is:

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') \right].$$

Equivalently, define the error $\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$ and update $Q(s, a) \leftarrow Q(s, a) + \alpha \delta$.

Worked example: old Q-value 2, reward 1, best next-state Q-value 4, discount 0.9, learning rate 0.5. The target is $1 + 0.9 \times 4 = 4.6$, so the new estimate is $0.5 \times 2 + 0.5 \times 4.6 = 3.3$.

The update changes the value of the state-action pair just experienced, not the value of the destination state. If the destination is terminal, its future value is zero and the target becomes the immediate reward.

Why the implementation details matter

Unseen actions must remain candidates. If one action has Q-value -5 and an untried action has the default value 0 , the untried action is currently preferred. Your methods enumerate all legal actions before maximizing, so they handle this case.

Ties are broken randomly. This avoids consistently favoring the first action solely because of list order. It also means epsilon zero does not imply that all choices are deterministic.

Q-values are accessed through `getQValue`. That allows a subclass to change the representation while reusing action selection. The optional approximate agent was designed to use this feature.

“For each transition, I moved the old Q-value toward the observed reward plus the discounted best estimated future value. The table defaults to zero, I include unseen legal actions in the maximum, and I break ties randomly.”

Why is Q-learning off-policy? The update target uses the greedy next-state maximum even when the actual behavior policy explores. It does not use the particular next action sampled by the exploratory policy.

7. Question 5 — Epsilon-greedy behavior and Crawler

What was required and how you implemented it

Complete `getAction` so that the agent explores with probability epsilon and otherwise exploits its current estimates. Your implementation:

1. Gets the legal actions and returns `None` if the list is empty.
2. Uses `util.flipCoin(self.epsilon)` to choose the exploration branch.
3. On exploration, calls `random.choice(legalActions)`.
4. Otherwise, calls `computeActionFromQValues(state)`.

Exploration chooses from **all** legal actions, including the best one. With four actions and one unique best action, the probability of choosing that best action is $(1 - \epsilon) + \epsilon/4$. At epsilon 0.1 , that is 0.925 .

Explaining the Gridworld experiments

With noise set to zero, differences between epsilon 0.1 and 0.9 mainly reflect action exploration. At 0.1 , the agent usually exploits what it has learned. At 0.9 , it spends most decisions choosing random actions, so even a useful Q-table can coexist with poor observed returns.

Q-values estimate returns under greedy continuation, while the training agent makes exploratory moves and starts with inaccurate estimates. Therefore, average training returns need not match the displayed values.

Why the same code runs Crawler

The learner uses legal actions, state identifiers, rewards and transitions through the common interface. It does not assume states are grid coordinates or that actions are compass directions. The provided Crawler environment supplies the task-specific states and rewards.

Your saved Crawler note proposes starting with roughly medium exploration, about 0.5 , and a medium learning rate, then lowering both once useful forward movement emerges.

“Early on, exploration helps the crawler discover useful movements, and a moderate learning rate lets it respond to experience. Later, reducing epsilon makes it repeat useful movements more consistently, while reducing alpha stabilizes its estimates.”

The note describes a strategy for changing parameters during training; your Q-learning code does **not** implement an automatic decay schedule. The GUI step delay controls simulation speed, not the Bellman update. Gamma controls how much future movement reward matters.

8. Question 6 — Bridge crossing with limited experience

What was required

Train on noiseless BridgeGrid for **50 episodes**, compare epsilon 1 and epsilon 0, and decide whether some epsilon/learning-rate pair makes learning the optimal policy more than 99% likely. The answer must not depend on a favorable action tie-breaking convention.

What your file returns

`analysis.question6()` returns 'NOT POSSIBLE'. The local epsilon 0.1 and learning-rate 0.5 assignments are unused because the tuple return is commented out.

The handout mistakenly refers to `question8()` in the Q6 text. Your file includes a second `question8()` returning the same string, but the Q6 autograder actually calls `question6()`. This extra analysis function does not implement optional approximate Q-learning.

A careful explanation

With epsilon zero, the nearby positive reward can make the learner settle on the short exit before discovering the distant reward. However, zero epsilon does not make discovery literally impossible: random tie-breaking can initially select different untried actions.

With large epsilon, exploration is more frequent, but random moves can also send the agent into the chasm. Noise zero only removes unintended movement; it does not remove bad actions chosen deliberately by the exploration branch. A useful crossing must be discovered and its reward must propagate backward through updates within the limited episode budget.

“My answer was NOT POSSIBLE for the question’s greater-than-99-percent reliability requirement after 50 episodes. Low exploration can lock the learner into the nearby reward, while high exploration creates many failed crossings. Changing the learning rate affects updates to observed experiences, but does not ensure the agent collects the experiences it needs.”

This explains the difficulty; it is not a formal probability proof. The Q6 autograder accepts the submitted answer, but that test checks the answer string rather than running a statistical reliability experiment. Do not turn “not reliably within this budget” into “Q-learning can never solve the bridge.”

An issue in the extra parameter-search script

`search_qlearning_params.py` loops over 500 episodes, tests 64 epsilon/alpha combinations across 100 seeds, then evaluates each run for 100 episodes. Its saved CSV’s best mean success rate is 0.07.

There is an implementation caveat: the script constructs `QLearningAgent` without setting `numTraining`. The inherited default is **100**. `gridworld.runEpisode()` calls `stopEpisode()`, which sets epsilon and alpha to zero after that limit. Consequently, in the current script, the final 400 of the nominal 500 training-loop episodes do not learn or explore.

Use those files only as supplementary exploration, not as proof of the exact 50-episode requirement or as evidence of 500 learning episodes. This guide identifies the issue without changing the script.

9. Question 7 — Applying Q-learning to Pacman

What was required

Reuse the learner for Pacman, train on `smallGrid`, then evaluate with learning and exploration disabled. The handout's example uses 2,000 training games and 10 test games; the autograder uses 100 test games after training.

How your implementation supports it

The provided `PacmanQAgent` inherits your Q-learning methods. Its defaults are epsilon 0.05, alpha 0.2 and gamma 0.8. Its `getAction` also calls `doAction`, allowing the framework to track the state/action for later reward updates.

After the requested training games, the base class sets:

```
self.epsilon = 0.0 # disable exploratory branch
self.alpha = 0.0 # freeze Q-value learning
```

Random tie-breaking can still occur. Turning off exploration does not require every equal-valued decision to be deterministic.

A Pacman state represents the board configuration, including food and ghost information; it is not just Pacman's coordinates. The transition incorporates the ghosts' response.

Your result

The saved `images/q7/results.png` shows **100 wins out of 100**, average score **500.68**, and Q7 **1/1**. A fresh local autograder run for this guide reproduced those results.

"The same generic Q-learning implementation worked in Pacman through the supplied subclass. After 2,000 training games, the recorded test run won all 100 games with learning and exploration disabled. That establishes performance for this test setup, not a universal win guarantee."

Why does tabular learning scale badly to larger layouts? Each distinct board configuration has separate Q-values. Experience with a ghost in one configuration does not automatically teach the same lesson for many similar configurations. The state space grows rapidly and most pairs receive too little experience.

10. Question 8 — Optional approximate Q-learning

This question was optional. `ApproximateQAgent`.`getQValue` and `update` still contain `util.raiseNotDefined()`. Do not describe them as completed work.

If asked about the idea, explain:

$$Q(s, a) = \sum_i w_i f_i(s, a), \quad w_i \leftarrow w_i + \alpha \delta f_i(s, a).$$

Features such as distance to food or nearby ghosts let different states share learned weights. The temporal-difference error has the same form as tabular Q-learning. This is useful conceptual context, not an implementation you submitted in these files.

11. Question 9(b) — CartPole policy search with REINFORCE

What was required

Implement a random baseline and a linear softmax policy, collect trajectories, compute the REINFORCE loss, and optimize the policy. Your selected experiment is `CartPole-v1`.

The observation has four components: cart position, cart velocity, pole angle and pole angular velocity. There are two actions: push left and push right. Rewards are +1 per step; a reward of 500 reaches the selected environment's episode-length cap.

The notebook's introductory prose contains inconsistent angle thresholds, so avoid quoting it as a verified environment specification. For the interview, the relevant point is that the episode ends on failure or at its time limit.

A. Baseline policies

`RandomPolicy.forward` returns uniform probabilities using:

```
torch.full((self.n_actions,), 1.0 / self.n_actions)
```

For `CartPole`, this is `[0.5, 0.5]`, independent of the observation. The policy itself returns probabilities; `run_episode` samples the action.

The supplied `NaivePolicy` is bang-bang control: if `state[2] < 0`, push left; otherwise push right. It reacts to the pole angle with one of two fixed actions and learns no parameters. It ignores velocities and cart position.

Your evaluation code deliberately continues sampling for `RandomPolicy`. Taking the argmax of `[0.5, 0.5]` would repeatedly select the first action and would no longer evaluate a random baseline.

The notebook does not preserve numerical training/evaluation comparisons for the random and naive policies. Explain their implementations without inventing baseline scores.

B. The learned policy

`LinearPolicy` consists of:

```
self.linear = nn.Linear(obs_dim, n_actions, bias=False)
self.softmax = nn.Softmax(dim=-1)
```

The linear layer maps the four observations to two scores, or logits. Softmax converts those scores to probabilities:

$$\pi_{\theta}(a | x) = \frac{\exp(w_a^T x)}{\sum_b \exp(w_b^T x)}.$$

There are **8 stored trainable weights**, from 4 inputs times 2 actions. `bias=False` matches the specified no-bias parameterization. There is no hidden layer. The scores are linear in the state, while softmax is nonlinear.

Unlike tabular Q-learning, this model directly represents action probabilities. It does not store a Q-table or learn a separate value function.

C. Collecting an episode

In `run_episode`:

1. Reset the environment and convert observations to float32 tensors.
2. Compute action probabilities and construct `Categorical(probs)`.
3. During training, sample an action. During learned-policy evaluation, choose the maximum-probability action.
4. Save `dist.log_prob(action)` with its computation graph intact.
5. Call `env.step(action.item())`, save the reward and continue until `terminated` or `truncated`.

`terminated` indicates an environment terminal condition, while `truncated` handles a cutoff such as the time limit. Keeping log probabilities as tensors allows `backward()` to compute derivatives with respect to the policy weights. Converting them to plain numbers would break that connection.

D. Reward-to-go and the loss

For each action, your `reinforce_loss` computes the return from its corresponding reward onward:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}, \quad \mathcal{L} = - \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t | x_t) G_t.$$

Here `rewards[t]` is the reward received **after** action `t`, so including it is correct. For rewards `[1, 1, 1]` and gamma 1, the returns are `[3, 2, 1]`.

The negative sign is essential because PyTorch optimizers minimize a loss, whereas policy gradient seeks to increase expected return. The update reinforces sampled actions in proportion to their observed future return. A large return does not prove a particular action was individually optimal; REINFORCE uses noisy trajectory samples.

The saved experiment uses **gamma 1.0**, matching the undiscounted episodic objective. Your function also computes discounted reward-to-go when passed a smaller gamma. The supported option should not be confused with the experiment actually recorded.

There is no learned baseline, advantage calculation, return normalization or critic. Summing every suffix separately costs quadratic time in episode length; a backward cumulative-return pass could reduce this, but the handout explicitly deprioritized efficiency.

E. Optimization

For the linear policy, each episode is followed by:

```
optimizer.zero_grad()
loss.backward()
optimizer.step()
scheduler.step()
```

The first line clears old gradients; the second computes gradients of this episode's loss; the third updates parameters; the fourth advances the learning-rate schedule. You make **one optimizer update per episode**, using all its steps.

You chose Adam with initial learning rate **0.01** and `StepLR(step_size=100, gamma=0.9)`. Every 100 optimizer updates, the learning rate is multiplied by 0.9. The scheduler's `gamma=0.9` is a decay multiplier, separate from the RL discount `gamma=1.0`.

F. Your experiment and recorded results

Setting	Saved value
Environment	CartPole-v1
Policy	linear
Training episodes	1,000
Initial Adam learning rate	0.01
RL discount	1.0
Evaluation	Mean of 5 episodes per checkpoint
Evaluation frequency	Every 100 loop iterations, starting after the first training episode

Training episode label	Mean greedy evaluation reward
1	9.2
101	311.4
201	311.2
301	500.0
401, 501, 601, 701, 801, 901	500.0 at every saved checkpoint

The evaluation starts at labels 1, 101, etc., because the condition is `episode % eval_freq == 0` with zero-based indexing. The last recorded checkpoint is 901, not 1,000. The evaluation plot uses checkpoint indices on its x-axis, not episode numbers.

Training performance can be worse than evaluation because training samples the stochastic policy and evaluation takes its argmax. For example, the recorded training episode 501 has reward 96 while the five-episode evaluation mean is 500. Greedy evaluation happened to perform better here; that is not a theorem about every stochastic policy.

Do not read this loss like classification loss. Longer successful episodes contain more terms and larger returns, so the loss magnitude can increase while control performance improves. Reward is the relevant performance measure.

“I trained a four-input, two-output softmax policy directly with REINFORCE. I sampled actions to collect episodes, weighted each saved log probability by reward-to-go, and minimized the negative sum using Adam. Greedy evaluation reached the 500-step cap at episode 301 and stayed there at all later recorded checkpoints.”

These are saved results from one run, not a new training run performed for this guide. No explicit random seeds or repeated-run statistics are recorded in the experiment. The video/setup cells use Colab/Linux assumptions such as `apt-get` and `/content/video`; they are not a ready-made Windows setup.

12. Verification and evidence

The local autograder was rerun on **5 October 2026**, using the current files and Python 3.14.2:

Question	Result
Q1	4/4
Q2	1/1
Q3	5/5
Q4	4/4
Q5	2/2
Q6	1/1
Q7	1/1; 100/100 wins; average score 500.68
Total for Q1–Q7	18/18

These are local test results, not a registered course grade. Q8 was not tested as completed work. The CartPole figures above come from the notebook’s saved output, which was inspected rather than retrained.

For revision, open these locations alongside this guide:

- `valueIterationAgents.py`: `runValueIteration`, `computeQValueFromValues`, `computeActionFromValues`.
- `analysis.py`: the exact tuples and the returned string in Q6.
- `qlearningAgents.py`: the Q-learning table, update and epsilon-greedy selection.
- `learningAgents.py`: `stopEpisode`, which disables learning and exploration.
- Notebook cells titled “Random agent class”, “Linear agent class”, “Run Episode”, and “Configure your experiment”.

The exercises originate from the UC Berkeley Pacman AI projects and the lab materials by Rubén Martínez Cantín, University of Zaragoza. This guide is for private interview preparation alongside those materials.

13. Final rehearsal

Practise answering these without reading the code first, then open the relevant method and point to the implementation.

Teacher's question	Core answer
What distinguishes value iteration from Q-learning?	Known-model expected backups versus sampled experience updates
Why a separate value table each sweep?	To preserve batch updates using the same previous values
How do discount, noise and living reward differ?	Future reward weighting, transition uncertainty and immediate step reward
Why can an unseen action be best?	Its default zero can exceed learned negative values
Is epsilon zero completely deterministic?	No; maximizing actions are still chosen with random tie-breaking
Why is Q-learning off-policy?	Its target uses the greedy maximum regardless of exploratory behavior
Why is your Q3(e) unusual?	It cycles under the checked noiseless path despite a negative living reward
What does NOT POSSIBLE mean in Q6?	The specific short-budget reliability requirement, not inability to ever learn
What changes between Pacman training and testing?	Epsilon and alpha become zero
Why does tabular Pacman struggle to scale?	Separate values for many board configurations, without generalization
How many CartPole parameters did you use?	Eight weights, with no bias or hidden layer
Why a negative REINFORCE loss?	Gradient descent on the negative objective performs policy-gradient ascent
Why save log probabilities?	To differentiate the likelihood of the sampled actions
Why can loss increase while reward improves?	Episode lengths and return weights change the loss scale
What did you actually demonstrate?	Local Q1–Q7 tests passed; saved CartPole evaluations reached 500 in one run

If time is short, prioritize explaining the Bellman update, the Q-learning update, epsilon versus noise, your Q3 parameter table, and the REINFORCE loss. Then rehearse the Q3(e) and Q6 caveats: these are the places where a precise explanation matters most.